

```
import numpy as np
```

```
def fuzzy_union(A, B):
    return np.maximum(A, B)
def fuzzy_intersection(A, B):
    return np.minimum(A, B)

def fuzzy_complement(A):
    return 1 - A

def fuzzy_difference(A, B):
    # Correct formula: Min(A, Complement of B)
    return np.minimum(A, 1 - B)

# --- Fuzzy Relation Operations ---
def cartesian_product(A, B):
    # Standard fuzzy relation uses Min
    return np.minimum.outer(A, B)

def max_min_composition(R, S):
    # R is (m x n), S is (n x p)
    # Result is (m x p)
    result = np.zeros((R.shape[0], S.shape[1]))
    for i in range(R.shape[0]):
        for j in range(S.shape[1]):
            result[i, j] = np.max(np.minimum(R[i, :], S[:, j]))
    return result
```

```
# --- Example Usage ---
A = np.array([0.2, 0.4, 0.6, 0.8])
B = np.array([0.3, 0.5, 0.7, 0.9])
print("Union:", fuzzy_union(A, B))
print("Intersection:", fuzzy_intersection(A, B))
print("Complement of A:", fuzzy_complement(A))

print("Difference (A - B):", fuzzy_difference(A, B))
```

```
Union: [0.3 0.5 0.7 0.9]
Intersection: [0.2 0.4 0.6 0.8]
Complement of A: [0.8 0.6 0.4 0.2]
Difference (A - B): [0.2 0.4 0.3 0.1]
```

```
R = np.array([[0.2, 0.5],
               [0.4, 0.7]])

S = np.array([[0.6, 0.3],
               [0.8, 0.5]])
```

```
# Cartesian Product of Sets A and B
```

```
rel_AB = cartesian_product(A, B)
print("\nCartesian Product (Set A x Set B):\n", rel_AB)

# Max-Min Composition of Relations R and S
comp_RS = max_min_composition(R, S)
print("\nMax-Min Composition (R ◦ S):\n", comp_RS)
```

Cartesian Product (Set A x Set B):

```
[[0.2 0.2 0.2 0.2]
 [0.3 0.4 0.4 0.4]
 [0.3 0.5 0.6 0.6]
 [0.3 0.5 0.7 0.8]]
```

Max-Min Composition (R ◦ S):

```
[[0.5 0.5]
 [0.7 0.5]]
```